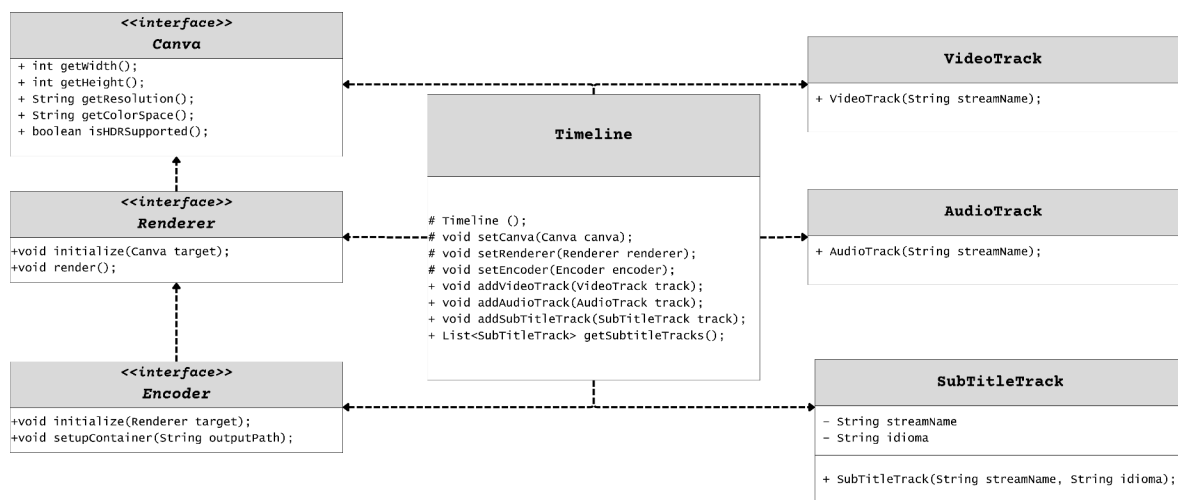


Sua equipe de desenvolvimento é responsável por construir um *framework* de edição e renderização de vídeo não-linear, flexível o suficiente para atender desde produtores de conteúdo a estúdios de cinema. A edição de vídeo não linear é um método de edição digital onde você pode acessar, modificar e ordenar qualquer quadro ou clipe de vídeo instantaneamente, sem precisar seguir uma ordem cronológica rígida ou alterar os arquivos originais.

Um fragmento do diagrama de classes proposto é apresentado abaixo:



Durante o processo de edição, a entidade central do domínio desse framework é a classe **Timeline**. Uma **Timeline** modela a dimensão temporal e estrutural do projeto de edição. Sua equipe de análise, definiu que a Timeline possui uma configuração rica e complexa, contendo propriedades espaciais de projeção gráfica (definidas por um **Canvas**), o motor de processamento visual para composição de efeitos (o **Renderer**), o algoritmo de compressão e exportação de arquivo final (o **Encoder**), além de listas ordenadas de trilhas de vídeo (**VideoTrack**), trilhas de áudio (**AudioTrack**) e trilhas de legenda (**SubTitleTrack**).

Estes outros elementos do framework são descritos a seguir:

- A Interface **Canvas** descreve a abstração da área de projeção visual onde o vídeo é renderizado e editado. Ele define as propriedades espaciais e geométricas da imagem, como a resolução nativa (ex: Full HD, 4K), a proporção de tela (ex: 16:9 horizontal, 9:16 vertical) e o espaço de cor (ex: Rec. 709, DCI-P3). É o "quadro" onde os elementos visuais são organizados.
- A Interface **Renderer** é o motor responsável por processar os dados brutos e transformá-los em imagens visíveis no Canvas. Ele lida com a computação gráfica em si: composição de múltiplas camadas de vídeo, aplicação de filtros, efeitos de transição, *color grading* e transformações geométricas (como escala e rotação).
- A Interface **Encoder** descreve o componente encarregado de pegar os quadros gerados pelo renderizador e compactá-los em um formato de arquivo ou fluxo de dados específico, utilizando um algoritmo de compressão (codec). Ele define como o sinal de vídeo e áudio serão estruturados

digitalmente para armazenamento ou transmissão, equilibrando a relação entre qualidade visual e tamanho de arquivo (ex: H.264, ProRes, AV1).

- A Classe **VideoTrack** descreve a abstração de uma camada temporal sequencial voltada exclusivamente para elementos visuais (como clipes de vídeo brutos, imagens estáticas e geradores de preenchimento).
- A Classe **AudioTrack** descreve o contêiner encarregado de sequenciar e organizar os fluxos de ondas sonoras (como diálogos gravados, efeitos sonoros e trilhas musicais) ao longo do tempo do projeto.
- A Classe **SubtitleTrack** descreve a abstração de uma linha temporal especializada no gerenciamento de dados de texto cronometrados e metadados linguísticos associados ao fluxo audiovisual.

O *framework* precisa ser flexível para suportar desde construções minimalistas (ex: uma timeline rápida apenas com uma trilha de vídeo para redes sociais) até configurações massivas de cinema (com dezenas de trilhas de áudio, metadados HDR e múltiplos efeitos).

O *framework* deseja fornecer a possibilidade de criação de Timelines que possam ser usados para edição em qualidade **Cinema** - usando, por padrão, uma instância de **Canva4K**, como **render**, um **HighPrecisionRenderer** e como **encoder** um **ProResEncoder** - ou em qualidade **Minimal** para uso em redes sociais - usando um **VerticalCanvas**, **AACEncoder** e **FastRenderer**.

### Questão I

Considerando o contexto do *framework*, e em particular, a classe **Timeline**, a equipe de arquitetura identificou um problema: O processo de montagem das trilhas e vinculação do **Canvas** e **Renderer** deve seguir uma ordem lógica rígida em tempo de execução para evitar objetos em estado inválido: o **Renderer** precisa ser inicializado com o **Canva**, depois o **Encoder** deve ser inicializado com o **Renderer** e só então o **Canva**, **Renderer** e **Encoder** são adicionados a **Timeline**. Além disso, o *framework* deve prover configurações padronizadas, mas passíveis de alteração durante a criação do objeto. Proponha um padrão de projeto que permita a criação de Timelines de acordo com os requisitos apresentados.

Demonstre o padrão fornecendo um modelo de construção de um CinemaTimeline (**Canva4K**, **HighPrecisionRenderer** e **ProResEncoder**, disponíveis na biblioteca do *framework*) em uma classe cliente.

### Questão II

Durante as sessões de teste com usuários do *framework*, identificou-se uma necessidade crítica de usabilidade: os editores frequentemente realizam "testes de edição" (versões alternativas ou ramificações do projeto). Um usuário precisa ser capaz de realizar um *fork* de sua **Timeline** atual para testar uma nova trilha sonora ou uma gradação de cor diferente em uma "Variante B", sem o risco de corromper ou alterar o trabalho já consolidado na "Variante A".

Contudo, as alterações, incrementos e mudanças dinâmicas em Timeline, impedem a criação de uma nova **Timeline** à partir do mecanismo da Questão I, dado que a linha do tempo pode conter centenas de referências a componentes previamente instanciados, modificados ou inseridos à posteriori. Sugira e implemente um padrão que demonstre o processo de *fork* de uma timeline, para por exemplo, a substituição da legenda para um stream com legendas em francês ("sous-titres.stt") -

Apenas é necessário, exibir o código que garante o *fork* e alteração das legendas. Os componentes (Canva, Renderer e Encoder) não são alteráveis através de *fork*.

Mostre como seria realizado um *fork* executando a alteração da legenda descrita acima numa classe cliente

### ATENÇÃO

- As soluções devem ser escritas em código JAVA
- Identifique claramente o padrão que será aplicado em cada questão, bem como quais são os papéis (Participantes) no padrão, assumidos pelas classes do projeto.